Principles of Programming Languages Design and Implementation

Builing a Hulk Interpreter and Compiler

Alejandro Piad Morffis

2026-03-10

Table of contents

Preface 6

Introduction 9

I The Frontend 11

1 Arithmetic Expressions 12

2 Strings and Builtins 13

3 Variables and Binding 14

4 Control Flow 15

5 Functions and Recursion 16

6 Objects and Classes 17

7 Type Checking and Inference 18

II The Backend 19

8 The BANNER Intermediate Representation (IR) 20

9 The Metal Sandbox 21

10 Stack Machine 22

11 Object Representation 23

12 Virtual Method Calls 24

13 Garbage Collector 25

14 Final Polish 26

Appendices 27

A The HULK Programming Language 27
A.1 HULK in a nutshell . . . 27
A.1.1 A didactic language . . . 27
A.1.2 An incremental language . . . 28
A.1.3 BANNER: Intermediate Representation . . . 28
A.2 Expressions . . . 29
A.2.1 Arithmetic expressions . . . 29
A.2.2 Strings . . . 30
A.2.3 Builtin math functions and constants . . . 30
A.2.4 Expression blocks . . . 31
A.3 Functions . . . 31
A.3.1 Inline functions . . . 31
A.3.2 Full-form functions . . . 32
A.4 Variables . . . 32
A.4.1 Multiple variables . . . 33
A.4.2 Scoping rules . . . 33
A.4.3 Expression block body . . . 34
A.4.4 The `let` return value . . . 34
A.4.5 Redefining symbols . . . 34
A.4.6 Destructive assignment . . . 35
A.4.7 Rules for naming identifiers . . . 35
A.5 Conditionals . . . 36
A.5.1 Expression blocks in conditionals . . . 37
A.5.2 Multiple branches . . . 37
A.6 Loops . . . 37
A.6.1 The `while` loop . . . 37
A.6.2 The `for` loop . . . 38
A.7 Types . . . 38
A.7.1 Declaring types . . . 39
A.7.2 Instantiating types . . . 39
A.7.3 Inheritance . . . 40
A.7.4 Polymorphism . . . 41
A.8 Type checking . . . 41
A.8.1 Typing variables . . . 42
A.8.2 Typing functions and methods . . . 42
A.8.3 Typing attributes and type arguments . . . 42
A.8.4 Type conforming . . . 43
A.8.5 Testing for dynamic types . . . 44
A.8.6 Downcasting . . . 44
A.9 Type inference . . . 45
A.9.1 Type inference vs type checking . . . 45

A.9.2 Type inference of expressions . . . 46
A.9.3 Type inference of symbols . . . 46
A.9.4 Examples of ad-hoc type inference . . . 47
A.9.5 A general strategy for type inference . . . 47
A.10 Protocols . . . 50
A.10.1 Defining protocols . . . 50
A.10.2 Implementing protocols . . . 50
A.10.3 Variance in protocol implementation . . . 51
A.10.4 Conforming with protocols . . . 51
A.11 Iterables . . . 51
A.11.1 Using iterables with the `for` loop . . . 52
A.11.2 Typing iterables . . . 53
A.11.3 Implementing collections . . . 54
A.12 Vectors . . . 54
A.12.1 Explicit syntax . . . 54
A.12.2 Implicit syntax . . . 55
A.12.3 Typing vectors . . . 55
A.13 Functors . . . 56
A.13.1 Implicit functor implementation . . . 57
A.13.2 Lambda expressions . . . 58
A.13.3 Typing functors . . . 59
A.14 Macros . . . 59
A.14.1 Defining macros . . . 61
A.14.2 Variable sanitization . . . 62
A.14.3 Symbolic arguments . . . 62
A.14.4 Variable placeholders . . . 63
A.14.5 Pattern matching . . . 64

B The BANNER Intermediate Representation 66
B.1 The Anatomy of a BANNER Program . . . 67
B.1.1 The `.TYPES` Section: Flattening the Hierarchy . . . 67
B.1.2 The `.DATA` Section: The Static Pool . . . 68
B.1.3 The `.CODE` Section: Procedural Execution . . . 68
B.2 The Instruction Set: A Minimalist Vocabulary . . . 69
B.3 Case Study: From HULK to BANNER . . . 71
B.4 Technical Deep Dive: "Everything is a Number" . . . 74
B.5 Conclusion: The Unseen Foundation . . . 75

C Tooling for HULK 76
C.1 Editor Support and Syntax Highlighting . . . 76
C.1.1 TextMate Grammars . . . 76
C.1.2 Quarto Integration . . . 76

C.2 Language Server Protocol (LSP) 76
C.2.1 Diagnostics 77
C.2.2 Navigation and Go to Definition 77
C.2.3 Hover Information 77
C.3 VS Code Extension Development 77

D The Instructor's Manual **78**

Preface

This book is primarily about making compilers, but it is also so much more. A compiler is one of the most exciting (and complex) projects you could attempt, and of the most interesting pieces of software you can examine. Building a compiler requires a combination of deep theoretical foundations, robust software engineering practices, and clever algorithm design and optimization. In a way, a compiler is the quintessential Computer Science application. This is why, in the process of building a compiler from scratch, you can learn a whole lot about many interrelated areas in Computer Science.

But why do we need compilers at all? You see, there is a large distance between the level of reasoning that occurs in the brain and the level of reasoning that occurs on a computer—at least, modern, traditional electronic computers like the one where you're reading this. Compilers are our best tools so far to bridge this gap. Here's why.

Problems in any domain are solved by thinking at a level of abstraction with a language that describes the rules of that domain. For example, if you're sending a rocket to the moon, you will think in terms of the physics and chemistry of rocket propulsion, the differential equations that model orbital mechanics, and the logistics and scheduling involved.

On the other hand, you have to explain all these things to a computer. And computers are very dumb. At their core, computers are just complex state machines that can do some basic arithmetics and move bits from one part of the memory to another. One of the most surprising insights in all of science is that, it turns out, this is all you need to be able to solve *any* solvable problem—an idea we will revisit in some detail in later chapters.

But let's go back to the core of the problem. The issue is that we have to deal with two widely different levels of abstractions: the higher level where you can talk and reason about rockets and planets and physics—the domain language—and the lower level where you have to talk and reason about bits and registers and arithmetic operations—the machine language.

There was a time when these levels of abstraction—these two languages— had to be connected by the programmer. In fact, at this time, the difference between analyst and programmer was precisely that the analyst designed the solution in his domain language, and the programmer translated it into an executable program in machine language. (This was, incidentally, also the time when women were mostly programmers and men mostly analysts, because many considered "programming" just a low-level translation task not worthy of intellectual pursuit. Oh, the irony! But I digress.)

Then, 1952, Grace Hooper came up with a brilliant idea. She was working on the simulation of ballistic trajectories. To direct a projectile to its target, physical models are described in a language of differential equations and Newtonian mechanics. However, in order to implement these models in a computing device, it is necessary to speak in a language of registers, stacks and interrupts.

This gap made programming extremely difficult, and slowed the development of new models extremely because at every step there could be errors in both modeling and coding. When something went wrong, whose fault was it? From the analyst or from the programmer? Or worse, the computer system?

But here is the kicker. Seeing that the process of converting differential equations into concrete programs was fundamentally mechanical–Hopper thought—why not let the computer itself do this conversion? And thus, the notion of a high level *programming language* was born!

The idea seems straightforward in hindsight: let's design a language that allows analysts to express their solutions to problems—their algorithms—as close as possible to the problem domain—e.g., using standard mathematical notation, functions, collections of numbers, and other relevant abstractions. Then, let's write another program that will translate this high level program into a low level *equivalent* program, taking care of all the complicated bits and registry manipulation, abstracting away the machine language so the analyst doesn't need to learn it at all.

This genius idea would take several years to perfect to the point of becoming a reality. Grace Hooper's first compiler for the A-0 language was actually practically a linker with some basic functions. The first high-level languages to have "serious" compilers are FORTRAN (1957, John Backus), ALGOL (1958, Friedrich Bauer), and COBOL (1960, Grace Hooper). An additional advantage, in addition to reducing development time, was the possibility of compiling the same program for multiple platforms. In 1960, for the first time, the same COBOL program was compiled for two different machines: UNIVAC II and RCA 501.

At this point the languages became sufficiently complicated, to the point that compilers could no longer be written "by hand." So it was necessary to turn to theory, and develop a science about what types of programming languages could be compiled, and with what compilers. This gave birth, in 1960, to the science that we know today as Compilation.

Motivated not only by a practical reason, but also based on the most solid theoretical principles, building compilers became one of the first justifications for Computer Science to question its own problems and limitations, and stop being seen as a mere calculation tool. Problems as distant as natural language processing and the nature of computable functions have fallen under the scope of the problems studied in this field. Today compilation is a solid science, founded on years of formal theory and engineering practice.

Hidden beneath all this formal apparatus and the full range of theoretical and practical experiences and results of the last 60 years, we can find a more fundamental question, a

question that perhaps goes back to Alan Turing himself, or even further, to Ada Lovelace and Charles Babbage with his analytical engine. The question is this:

How to talk to a computer?

All attempts to design languages, all algorithms and techniques discovered, all design patterns and architectures, are ultimately tied to the desire to be able to ask a *question* to the computer, and get an *answer* in return. It doesn't matter if the question is to calculate a certain projectile trajectory, or to find the sequence of parameters that minimize a certain function. Every program is in a way a conversation with the computer, a communication channel, which we want to be powerful enough to be able to express our most complex ideas, and simple enough to be understood by a Turing machine. As we will see in this book, finding the right balance is an extremely interesting problem, and trying to answer it will take us down a path that will raise many other questions, including the following:

- What types of languages is a computer capable of *understanding*?
- How much of a language must be *understood* in order to have a conversation?
- What is *understanding* a language?
- Is it as easy or difficult to *understand* as to *speak* a language?
- Can we characterize languages in computational terms according to their complexity to be *understood* by a computer?
- How are these languages related to human language?
- What can we learn about the nature of computers and computable problems from the languages they are able to recognize?
- What can we learn about human language to make computers smarter?
- What can we learn about human language, and the very nature of our intelligence, from studying languages understandable by different types of machines?

These questions, although not all will be directly answered in the following chapters, form the backbone of the book content, in the sense that everything presented is with the intention of, at least, shedding a little light on these topics. We hope that at the end of the book, students will be able to discuss the philosophical implications of the possible answers to these questions, and not just the technical or more practical issues that the book attacks. For this reason, we will try as far as possible, in addition to the technical content, to occasionally add some comments or more philosophical discussions regarding these and similar questions.

So this book is primarily about making compilers. But it is also about some of most profound questions in Computer Science and some of the most surprising answers—including, the lack of answers for many of these questions.

Introduction

We will begin this journey by dissecting the canonical computational system of formal language theory: a compiler. Broadly speaking, a compiler is nothing more than a program, whose input and output also happen to be programs. The input is a program in a language that we will call "high level", and the output in a "low level" language, which is equivalent to the first. Exactly what is high and low will depend on many factors, and there is no formal definition. In general, a high-level language is one that is comfortable for us as programmers to express the operations that we are interested in executing. Likewise, a low-level language is one that a computing device can execute efficiently. Perhaps the most typical examples are an object-oriented language and an assembly language respectively, but there are many other input and output language combinations of interest.

Now, before diving headlong into the anatomy of a compiler, it is worth mentioning some related language processing systems. We can try to categorize them according to the "type" of the input and output language. First of all, the classic example is when we want to convert a high-level language to a low-level language, and we call this system a **compiler**. The opposite case, when we want to convert from a low-level language to a high-level language, we can call it a **decompiler** by analogy. These types of tools are useful for analyzing and reverse engineering programs for which, perhaps, we no longer have the source code, and we need to understand or modify. The other two cases, high level to high level and low level to level are basically **translators**; and sometimes they are also called **transpilers**. For example, TypeScript is a high-level language that "transpiles" to JavaScript, another high-level language. Among low-level languages we can also have translators. An example is the so-called **JIT** (*just-in-time*) compilers, which are used to translate a program compiled in a generic low-level language (for example **.NET IL**) into a machine language. specific to the architecture where it is executed.

Let us then return to the classic case, the **compiler**. In this course we are going to use as a teaching guide the design of a compiler for the HULK language, which will compile to a machine language called MIPS. Details of both languages will be introduced as appropriate, but for now it can be said that HULK is an object-oriented language, with automatic garbage collection, simple inheritance, polymorphism, and a unified type system. MIPS is a stack assembly language for a 32-bit architecture with registers and arithmetic, logic, and string-oriented operations.

Let us then try to define this machinery step by step. Abstractly our compiler is a "black box" that converts programs written in HULK to programs written in MIPS:

To begin to uncover this black box, let's notice that we have at least two independent components: one that operates in HULK language and another that operates in MIPS language. We need to be able to "read" a program in HULK and "write" it in MIPS. We will call the first module, which "reads", *parser*, or syntax analyzer, for historical reasons that we will see later. We will simply call the second component the *generator*.

From here a question immediately arises: what communication protocol do these modules have? It is necessary to design a kind of intermediate language, a representation mechanism that is neither HULK nor MIPS, but something that is "halfway" between the two. That is, it is necessary to translate the HULK program into some form of abstract representation, independent of the syntax, which can then be interpreted by the generator and written in MIPS. Let's call it *intermediate representation (IR)* for now.

Part I

The Frontend

1 Arithmetic Expressions

We begin our journey by building the core of any expression language: arithmetic. We will implement a lexer, a parser, and a tree-walking interpreter capable of evaluating basic mathematical operations.

```
# Lexer implementation for HULK
```

```
# Parser implementation for HULK
```

2 Strings and Builtins

In this chapter, we extend HULK with string literals and the concatenation operator. We also introduce the concept of builtin functions, allowing our language to interact with the outside world via `print` and mathematical constants like `PI`.

```
# Builtin functions and constants for HULK
```

3 Variables and Binding

Variables allow us to store and reuse values. We will implement lexically-scoped variables using the `let` expression and manage local environments in our interpreter.

```
# Scope and Environment management for HULK
```

4 Control Flow

Decisions are fundamental to programming. We introduce boolean literals, comparison operators, and the `if-then-else` expression, which in HULK is itself an expression with a return value.

```
# Evaluator updates for Control Flow
```

5 Functions and Recursion

We move beyond simple expressions to define reusable logic. We will implement global function definitions, parameter passing, and recursive calls, which are essential for complex algorithms.

```
# Function management for HULK
```

6 Objects and Classes

HULK is an object-oriented language. We will implement class definitions, attribute storage, and method invocation, laying the groundwork for more advanced abstraction.

```
# Type and Object system for HULK
```

7 Type Checking and Inference

Before code generation, we must ensure its correctness. We will build a semantic analyzer that verifies types and performs type inference to keep our HULK programs safe.

```
# Semantic analyzer and Type inference for HULK
```

Part II

The Backend

8 The BANNER Intermediate Representation (IR)

We bridge the gap between high-level HULK and low-level machine code. We define the BANNER Intermediate Representation (IR) and implement a "lowering" pass to transform our AST into a linear sequence of instructions.

```
# Transpiler from HULK to Banner IR
```

9 The Metal Sandbox

We transition to the Rust ecosystem. This chapter sets up the backend project, introduces Rust's memory safety model, and prepares the environment for building our high-performance VM.

```rust
fn main() {
    println!("Hello from the Metal Sandbox");
}
```

10 Stack Machine

We implement the heart of our backend: a stack-based virtual machine. We will define the instruction set and the execution loop that processes BANNER IR at high speeds.

```
// Stack Machine implementation
```

11 Object Representation

High-level objects need a low-level home. We design the memory layout for strings, class instances, and arrays, and implement a heap to manage their lifecycle.

```
// Heap and Object representation
```

12 Virtual Method Calls

Supporting polymorphism requires dynamic dispatch. We implement virtual method tables (vtables) and the runtime logic needed to resolve method calls on objects.

```
// VTable and Dynamic Dispatch
```

13 Garbage Collector

Automatic memory management is a key feature of modern languages. We will implement a garbage collector to reclaim unused memory in our VM, ensuring our programs can run indefinitely without leaks.

```
// Garbage Collector implementation
```

14 Final Polish

We bring everything together. In this final chapter, we refine the CLI, add the remaining standard library functions, and reflect on the journey of building a complete language from scratch.

```
// CLI and Final Integration
```

A The HULK Programming Language

In this final part of the book, we present a straightforward and comprehensive definition of the HULK programming language, along with a set of possible extensions. This part formalizes the language we've been working on the entire book, and should serve as a reference for anyone wanting to build their own HULK compiler.

A.1 HULK in a nutshell

HULK (**H**avana **U**niversity **L**anguage for **K**ompilers) is a didactic, type-safe, object-oriented and incremental programming language, designed for the course Introduction to Compilers in the Computer Science major at University of Havana.

A simple "Hello World" in HULK looks like this:

```
print("Hello World");
```

In a bird's eye view HULK is an object-oriented programming language, with simple inheritance, polymorphism, and encapsulation at the class level. Also, in HULK it is possible to define global functions outside the scope of all classes. It is also possible to define a *single global expression* that constitutes the entry point to the program.

Most of the syntactic constructions in HULK are expressions, including conditional instructions and cycles. HULK is a statically typed language with optional type inference, which means that some (or all) parts of a program can be annotated with types, and the compiler will verify the consistency of all operations.

A.1.1 A didactic language

The HULK language has been designed as a mechanism for learning and evaluating a college course about compilers. For this reason, certain language design decisions respond more to didactic questions than to theoretical or pragmatic questions. An illustrative example is the inclusion of a single basic numerical type. In practice, programming languages have several numeric types (`int`, `float`, `double`, `decimal`) to cover the wide range of trade-off between efficiency and expressivity. However, from the didactic point of view, it is enough complexity

to have to deal with a numerical type, and the inclusion of others does not bring anything new from our point of view.

Another important decision is the static typing with type inference, which will be explained later in detail. The motivation behind this feature is to allow students to first implement an evaluator for the language, and then worry about type verification. Likewise, the decision to have global expressions, global functions, and classes, responds to the need to introduce the various elements of language little by little. By having global expressions, it is possible to implement an expression interpreter without the need to solve context-sensitive problems. Later, students can implement functions and finally the object-oriented features. In this way students can learn on the fly as they add characteristics to the language, always having a valid subset of the language implemented.

A.1.2 An incremental language

As its name indicates, HULK is a huge language. Actually, the HULK language really is not really a single programming language, but a set of programming languages. That is, HULK is designed as a set of layers, each with a new language feature that add increasingly more complex functionalities on top of the previous layers. It starts with a basic syntax for expressions, then global functions, and then a unified type system with simple inheritance. Afterwards, HULK grows to contain arrays, delegates, type inference, iterators, among other characteristics. All these language features have been designed to be compatible with each other. Furthermore, each language feature clearly describes on which other language features it depends.

This design has been conceived to allow the use of HULK at a wide range of learning levels. As a language of expressions and functions, it is useful for introductory courses on parsing and basic compilation techniques. Object orientation introduces a whole universe of semantic complexities; however, the HULK type system is simple enough to illustrate the most common problems in semantic type verification. Vectors introduce problems related to memory management, while anonymous functions and iterators are fundamentally problems of transpilation and code generation. The inference of types and the verification of null-safety is an exercise in logical inference, which can be used in advanced courses. The idea is that each course defines its objectives of interest, and can use an appropriate subset of HULK to illustrate and evaluate them.

A.1.3 BANNER: Intermediate Representation

Even though HULK can be defined without specific compilation details, we also provide a didactic 3-address code for intermediate representation that is convenient to use with HULK. For obvious reasons, it's called BANNER – **B**asic 3-**A**dress li**N**ear i**N**t**E**mediate **R**epresentation.

A.2 Expressions

HULK is ultimately an expression-based language. Most of the syntactic constructions in HULK are expressions, including the body of all functions, loops, and any other block of code.

The body of a program in HULK always ends with a single global expression (and, if necessary, a final semicolon) that serves as the entrypoint of the program. This means that, of course, a program in HULK can consist of just one global expression.

For example, the following is a valid program in HULK:

```
42;
```

Obviously, this program has no side effects. A slightly more complicated program, probably the first one that does something, is this:

```
print(42);
```

In this program, `print` refers to a builtin function that prints the result of any expression in the output stream. We will talk about functions in a later section.

The rest of this section explains the basic expressions in HULK.

A.2.1 Arithmetic expressions

HULK defines three types of literal values: **numbers**, **strings**, and **booleans**. We will leave strings and booleans for later.

Numbers are 32-bit floating-point and support all basic arithmetic operations with the usual semantics: `+` (addition), `-` (subtraction), `*` (multiplication), ‘`(floating-point division),^`‘ (power), and parenthesized sub-expressions.

The following is a valid HULK program that computes and prints the result of a rather useless arithmetic expression:

```
print((((1 + 2) ^ 3) * 4) / 5);
```

All usual syntactic and precedence rules apply.

A.2.2 Strings

String literals in HULK are defined within enclosed double-quotes (`"`), such as in:

```
print("Hello World");
```

A double-quote can be included literally by escaping it:

```
print("The message is "Hello World"");
```

Other escaped characters are for line endings, and for tabs.

Strings can be concatenated with other strings (or the string representation of numbers) using the `@` operator:

```
print("The meaning of life is " @ 42);
```

A.2.3 Builtin math functions and constants

Besides `print`, HULK also provides some common mathematical operations encapsulated as builtin functions with their usual semantics. The list of builtin math functions is the following:

- `sqrt(<value>)` computes the square root if a value.
- `sin(<angle>)` computes the sine of an angle in radians.
- `cos(<angle>)` computes the cosine of an angle in radians.
- `exp(<value>)` computes the value of `e` raised to a value.
- `log(<base>, <value>)` computes the logarithm of a value in a given base.
- `rand()` returns a random uniform number between 0 and 1 (both inclusive).

Besides these functions, HULK also ships with two global constants: `PI` and `E` which represent the floating-point value of these mathematical constants.

As expected, functions can be nested in HULK (provided the use of types is consistent, but so far all we care about is functions from numbers to numbers, so we can forget about types until later on). Hence, the following is a valid HULK program.

```
print(sin(2 * PI) ^ 2 + cos(3 * PI / log(4, 64)));
```

More formally, function invocation is also an expression in HULK, so everywhere you expect an expression you can also put a call to builtin function, and you can freely mix arithmetic expressions and mathematical functions, as you would expect in any programming language.

A.2.4 Expression blocks

Anywhere an expression is allowed (or almost), you can also use an expression block, which is nothing but a series of expressions between curly braces (`{` and `}`), and separated by `;`.

The most trivial usage of expression blocks is to allow multiple `print` statements as the body of a program. For example, the following is a valid HULK program:

```
{
    print(42);
    print(sin(PI/2));
    print("Hello World");
}
```

When you use an expression block instead of a single expression, it is often not necessary to end with a semicolon (`;`), but it is not erroneous to do so either.

A.3 Functions

HULK also lets you define your own functions (of course!). A program in HULK can have an arbitrary number of functions defined before the final global expression (or expression block).

A function's body is always an expression (or expression block), hence all functions have a return value (and type), that is, the return value (and type) of its body.

A.3.1 Inline functions

The easiest way to define a function is the inline form. Here's an example:

```
function tan(x) => sin(x) / cos(x);
```

An inline function is defined by an identifier followed by arguments between parenthesis, then the `=>` symbol, and then a simple expression (not an expression block) as body, ending in `;`.

In HULK, all functions must be defined before the final global expression. All these functions live in a single global namespace, hence it is not allowed to repeat function names. Similarly, there are no overloads in HULK (at least in "basic" HULK).

Finally, the body of any function can use other functions, regardless of whether they are defined before or after the corresponding function. Thus, the following is a valid HULK program:

```
function cot(x) => 1 / tan(x);
function tan(x) => sin(x) / cos(x);

print(tan(PI) ** 2 + cot(PI) ** 2);
```

And of course, inline functions (and any other type of function) can call themselves recursively.

A.3.2 Full-form functions

Since inline functions only allow for a single expression as body (as complex as that may be), HULK also allows full-form functions, in which the body is an expression block.

Here's an example of a rather useless function that prints 4 times:

```
function operate(x, y) {
    print(x + y);
    print(x - y);
    print(x * y);
    print(x / y);
}
```

Note that the following form is discouraged for stylistic reasons:

```
function id(<args>) => {
    // ...
}
```

That is, you should either use the inline form with `=>` and a simple expression, or the full form with `{}` and an expression block.

A.4 Variables

Variables in HULK are lexically-scoped, which means that their scope is explicitely defined by the syntax. You use the `let` expression to introduce one or more variables and evaluate an expression in a new scope where those variables are defined.

The simplest form is introducing a single variable and using a single expression as body.

```
let msg = "Hello World" in print(msg);
```

Here `msg` is a new symbol that is defined *only* within the expression that goes after `in`.

A.4.1 Multiple variables

The `let` expression admits defining multiple variables at once like this:

```
let number = 42, text = "The meaning of life is" in
    print(text @ number);
```

This is semantically equivalent to the following long form:

```
let number = 42 in
    let text = "The meaning of life is" in
        print(text @ number);
```

As you can notice, `let` associates to the right, so the previous is also equivalent to:

```
let number = 42 in (
    let text = "The meaning of life is" in (
            print(text @ number)
        )
    );
```

A.4.2 Scoping rules

Since the binding is performed left-to-right (or equivalently starting from the outer let), and every variable is effectively bound in a new scope, you can safely use one variable when defining another:

```
let a = 6, b = a * 7 in print(b);
```

Which is equivalent to (and thus valid):

```
let a = 6 in
    let b = a * 7 in
        print(b);
```

A.4.3 Expression block body

You can also use an expression block as the body of a `let` expression:

```
let a = 5, b = 10, c = 20 in {
    print(a+b);
    print(b*c);
    print(c/a);
}
```

As we said before, semicolons (`;`) are seldom necessary after an expression block, but they are never wrong.

A.4.4 The `let` return value

As with almost everything in HULK, `let` is an expression, so it has a return value, which is obviously the return value of its body. This means the following is a valid HULK program:

```
let a = (let b = 6 in b * 7) in print(a);
```

Or more directly:

```
print(let b = 6 in b * 7);
```

This can be of course nested ad infinitum.

A.4.5 Redefining symbols

In HULK every new scope hides the symbols from the parent scope, which means you can redefine a variable name in an inner `let` expression:

```
let a = 20 in {
    let a = 42 in print(a);
    print(a);
}
```

The previous code prints `42` then `20`, since the inner `let` redefines the value of `a` inside its scope, but the value outside is still the one defined by the outer `let`.

And because of the scoping rules, the following is also valid:

```
let a = 7, a = 7 * 6 in print(a);
```

Which is equivalent to:

```
let a = 7 in
    let a = 7 * 6 in
        print(a);
```

A.4.6 Destructive assignment

Most of the time in HULK you won't need to overwrite a variable, but there are cases where you do. In those cases, you can use the destructive assignment operator `:=`, like this:

```
let a = 0 in {
    print(a);
    a := 1;
    print(a);
}
```

The previous program prints `0` and then `1`, since the value of `a` is overwritten before the second `print`. This is the **only** way in which a variable can be written to outside of a `let`.

As you would expect, the `:=` operator defines an expression too, which returns the value just assigned, so you can do the following:

```
let a = 0 in
    let b = a := 1 in {
        print(a);
        print(b);
    };
```

This is useful if you want to evaluate a complex expression to both test it (e.g, to se if its greater than zero) and store it for later use.

A.4.7 Rules for naming identifiers

Variables (and identifiers in general) in HULK can be named with any sequence of alphanumeric characters, plus underscore `_`, but must *always* begin with a letter (not a digit or `_`), hence the following are all valid identifiers:

- `x`
- `x0`
- `x_0`
- `lowercase`
- `TitleCase`
- `snake_case`
- `camelCase`

The following are invalid HULK identifiers:

- `_x`
- `x+y`
- `some method`
- `8ball`

And many others of course!

Since starting with an underscore `_` is invalid in user-produced HULK code, you will notice that when we talk about transpilation in HULK, variables and identifiers in transpiled code always start with `_`.

A.5 Conditionals

The `if` expression allows evaluating different expressions based on a condition.

```
let a = 42 in if (a % 2 == 0) print("Even") else print("odd");
```

Since `if` is itself an expression, returning the value of the branch that evaluated true, the previous program can be rewritten as follows:

```
let a = 42 in print(if (a % 2 == 0) "even" else "odd");
```

Conditions are just expressions of boolean type. The following are the valid boolean expressions:

- Boolean literals: `true` and `false`.
- Arithmetic comparison operators: `<`, `>`, `<=`, `>=`, `==`, `!=`, with their usual semantics.
- Boolean operators: `&` (and), `|` (or), and `!` (not) with their usual semantics.

A.5.1 Expression blocks in conditionals

The body of the `if` or the `else` part of a conditional (or both) can be an expression block as well:

```
let a = 42 in
    if (a % 2 == 0) {
        print(a);
        print("Even");
    }
    else print("Odd");
```

A.5.2 Multiple branches

The `if` expression supports multiple branches with the `elif` construction, which introduces another conditioned branch:

```
let a = 42, let mod = a % 3 in
    print(
        if (mod == 0) "Magic"
        elif (mod % 3 == 1) "Woke"
        else "Dumb"
    );
```

A.6 Loops

HULK defines two kinds of loops, the `while` expression and the `for` expression. Both loop constructions are expressions, returing the value of the

A.6.1 The `while` loop

A `while` loop evaluates a condition and its body while the condition is true. The body can be a simple expression or an expression block.

```
let a = 10 in while (a >= 0) {
    print(a);
    a := a - 1;
}
```

Since the return value of the `while` loop is the return value of its expression body, it can often be used directly as the body of a function.

```
function gcd(a, b) => while (a > 0)
    let m = a % b in {
        b := a;
        a := m;
    };
```

A.6.2 The `for` loop

A `for` loop iterates over an *iterable* of elements of a certain type. We will talk about iterables later on, but for now it suffices to say that if some expression evaluates to a collection, then the `for` loop can be used to iterate it.

For example, the builtin `range(<start>, <end>)` function evaluates to an iterable of numbers between `<start>` (inclusive) and `<end>` (non-inclusive).

```
for (x in range(0, 10)) print(x);
```

The `for` loop is semantically and operationally equivalent to the following:

```
let iterable = range(0, 10) in
    while (iterable.next())
        let x = iterable.current() in
            print(x);
```

In fact, what the reference implementation of the HULK compiler does in `for` loops is to transpile them into their `while` equivalent. This also effectively means that, just like the `while` loop, the `for` loop returns the last value of its body expression.

A.7 Types

HULK is ultimately an object-oriented language with simple inheritance and nominal typing. It also has features of structural typing via protocols, which support language features such as iterables, which we will explain later.

This section explains the basics of HULK's nominal typing system.

A type in HULK is basically a collection of attributes and methods, encapsulated under a type name. Attributes are always private, which means they can't be read or writen to from any code outside the type in which they are defined (not even inheritors), while methods are always public and virtual.

A.7.1 Declaring types

A new type is declared using the `type` keyword followed by a name, and a body composed of attribute definitions and method definitions. All attributes must be given an initialization expression. Methods, like functions, can have a single expression or an expression block as body;

```
type Point {
    x = 0;
    y = 0;

    getX() => self.x;
    getY() => self.y;

    setX(x) => self.x := x;
    setY(y) => self.y := y;
}
```

The body of every method is evaluated in a namespace that contains global symbols plus an especial symbol named `self` that references the current instance. The `self` symbol is **not** a keyword, which means it can be hidden by a `let` expression, or by a method argument.

However, when referring to the current instance, `self` is not a valid assignment target, so the following code should fail with a semantic error:

```
type A {
    // ...
    f() {
        self := new A(); // <-- Semantic error, `self` is not a valid assignment target
    }
}
```

A.7.2 Instantiating types

To instantiate a type you use the keyword `new` followed by the type name:

```
let pt = new Point() in
    print("x: " @ pt.getX() @ "; y: " @ pt.getY());
```

As you can see, type members are accessed by dot notation (`instance.member`).

You can pass arguments to a type, that you can use in the initialization expressions. This achieves an effect similar to having a single constructor.

```
type Point(x, y) {
    x = x;
    y = y;

    // ...
}
```

Then, at instantiation time, you can pass specific values:

```
let pt = new Point(3,4) in
    print("x: " @ pt.getX() @ "; y: " @ pt.getY());
```

Each attribute initialization expression is evaluated in a namespace that contains the global symbols and the type arguments, but no the `self` symbol. This means you cannot use other attributes of the same instance in an attribute initialization expression. This also means that you cannot assume any specifc order of initialization of attributes.

A.7.3 Inheritance

Types in HULK can inherit from other types. The base of the type hierarchy is a type named `Object` which has no public members, which is the type you implicitely inherit from by default. To inherit from a specific type, you use the `inherits` keyword followed by the type name:

```
type PolarPoint inherits Point {
    rho() => sqrt(self.getX() ^ 2 + self.getY() ^ 2);
    // ...
}
```

By default, a type inherits its parent type arguments, which means that to construct a `PolarPoint` you have to pass the `x` and `y` that `Point` is expecting:

```
let pt = new PolarPoint(3,4) in
    print("rho: " @ pt.rho());
```

If you want to define a different set of type arguments, then you have to provide initialization expressions for the parent type at the declaration:

```
type PolarPoint(phi, rho) inherits Point(rho * sin(phi), rho * cos(phi)) {
    // ...
}
```

During construction, the expressions for type arguments of the parent are evaluated in a namespace that contains global symbols plus the type arguments of the inheritor. Like before, you cannot assume a specific order of evaluation.

In HULK, the three builtin types (`Number`, `String`, and `Boolean`) implicitely inherit from `Object`, but it is a semantic error to inherit from these types.

A.7.4 Polymorphism

All type methods in HULK are virtual by definition, and can be redefined by an inheritor provided the exact same signature is used:

```
type Person(firstname, lastname) {
    firstname = firstname;
    lastname = lastname;

    name() => self.firstname @@ self.lastname;
}
```

> **NOTE**: `@@` is equivalent to `@ "  " @`. It is a shorthand to insert a whitespace between two concatenated strings. There is no `@@@` or beyond, we're not savages.

```
type Knight inherits Person {
    name() => "Sir" @@ base();
}

let p = new Knight("Phil", "Collins") in
    print(p.name()); // prints 'Sir Phil Collins'
```

The `base` symbol in every method refers to the implementation of the parent (or the closest ancestor that has an implementation).

A.8 Type checking

HULK is a statically-typed language with optional type annotations. So far you haven't seen any because HULK has a powerful type inference system which we will talk about later on. However, all symbols in HULK have a static type, and all programs in HULK are statically checked during compilation.

Tye annotations can be added anywhere a symbol is defined, that is:

- in variable declarations with `let` expressions;

- in function or method arguments and return type;
- in type attributes; and,
- in type arguments.

Let's see an example of each case.

A.8.1 Typing variables

Variables can be explicitely type-annotated in `let` expressions with the following syntax:

```
let x: Number = 42 in print(x);
```

The type checker will verify that the type inferred for the initialization expression is compatible with (formally, conforms to) the annotated type.

A.8.2 Typing functions and methods

All or a subset of a function's or method's arguments, and its return value, can be type-annotated with a similar syntax:

```
function tan(x: Number): Number => sin(x) / cos(x);
```

On the declaration side, the type checker will verify that the body of the method uses the types in a way that is consistent with their declaration. The exact meaning of this consistency is defined in the section about type semantics. The type checker will also verify that the return type of the body conforms to the annotated return type.

On the invocation side, the type checker will verify that the values passed as parameters conform to the annotated types.

Inside methods of a type `T`, the implicitly defined `self` symbol is always assumed as if annotated with type `T`.

A.8.3 Typing attributes and type arguments

In type definitions, attributes and type arguments can be type-annotated as follows:

```
type Point(x: Number, y: Number) {
    x: Number = x;
    y: Number = y;

    // ...
}
```

The type checker will verify that type arguments are used consistently inside attribute initialization expressions, and that the inferred type for each attribute initialization expression conforms to the attribute annotation.

A.8.4 Type conforming

The basic type relation in HULK is called *conforming* (`<=`). A type `T1` is said to *conform to* to another type `T2` (writen as `T1 <= T2`) if a variable of type `T2` can hold a value of type `T1` such that every possible operation that is semantically valid with `T2` is guaranteed to be semantically valid with `T1`.

In general, this means that the type checker will verify that the inferred type for any expression conforms to the corresponding type declared for that expression (e.g., the type of a variable, or the return type of a function).

The following rules provide an initial definition for the *conforming* relationship. The formal definition is given in the section about type semantics.

- Every type conforms to `Object`.
- Every type conforms to itself.
- If `T1` inherits `T2` then `T1` conforms to `T2`.
- If `T1` conforms to `T2` and `T2` conforms to `T3` then `T1` conforms to `T3`.
- The only types that conform to `Number`, `String`, and `Boolean`, are respectively those same types.

Types in HULK form a single hierarchy rooted at `Object`. In this hierarchy the *conforming* relationship is equivalent to the *descendant* relationship. Thus, if `T1` conforms to `T2` that means that `T1` is a descendant of `T2` (or trivially the same type). Thus, we can talk of the lowest common ancestor of a set of types `T1`, `T2`, …, `Tn`, which is the most specific type `T` such that all `Ti` conform to `T`. When two types are in different branches of the type hierarchy, they are effectively incomparable.

> **NOTE**: this conforming relationship is extended when we add protocols.

A.8.5 Testing for dynamic types

The `is` operator allows to test an object to check whether its dynamic type conforms to a specific static type.

```
type Bird {
    // ...
}

type Plane {
    // ...
}

type Superman {
    // ...
}

let x = new Superman() in
    print(
        if (x is Bird) "It's bird!"
        elif (x is Plane) "It's a plane!"
        else "No, it's Superman!"
    );
```

In general, before the `is` operator you can put any expression, not just a variable.

A.8.6 Downcasting

You can use the `as` operator to downcast an expression to a given static type. The result is a runtime error if the expression is not a suitable dynamic type, which means you should always test if you're unsure:

```
type A {
    // ...
}

type B inherits A {
    // ...
}

type C inherits A {
```

```
    // ...
}

let x : A = if (rand() < 0.5) new B() else new C() in
    if (x is B)
        let y : B = x as B in {
            // you can use y with static type B
        }
    else {
        // x cannot be downcasted to B
    }
```

A.9 Type inference

Since every program in HULK is statically type-checked, and type annotations are optional in most cases, this means that HULK infers types for most of the symbols in a program.

Because the problem of type inference is computationally complex, and ultimately unsolvable in the general case, the HULK reference definition doesn't give precise semantics about how the type inferer must work. Rather, we will give only a set of minimal constraints that the type inferer must assert if a type is inferred at all for a given symbol, or otherwise it must fail to infer types.

A.9.1 Type inference vs type checking

The type inferer works before the type checker, and assigns type annotations to all symbols that are not explicitly annotated, and to all the expressions. Afterwards, the type checker verifies that all semantic rules are valid.

Thus, even if a program is fully annotated, the type inferer still needs to work, since it needs to infer the type of all expressions. When some symbols are not explicitly annotated, the type inferer must also assign types for them.

Hence, there are two different moments when a semantic error can be reported. First, if the type inferer cannot infer the type of some symbol, a semantic error will be thrown to indicate the programmer that some symbol must be explicitly typed. Second, if the type inferer finished without errors, the type checker will verify that all types are consistent, and will report a semantic error if there is some incompatibilty.

A.9.2 Type inference of expressions

The first task of the type inferer is to infer the runtime type of any expression that appears in a HULK program. This process is performed bottom-up, starting from atomic sub-expressions (e.g., literals) and working up the AST. The exact rules for type inference of expressions is given in the section a'bout type semantics, but an intuitive introduction can be given at this point.

Literals are the easiest to type-infer, because their type comes directly from the parser. Arithmetic expressions are also easy, because their type is always `Number`. Likewise, string and boolean operators are straightforward.

The type of complex expressions that have an expression body is determined by the type of the body. This is the case of `let`, `while`, and `for`. The type of an expression block is the type of the last expresion of the block. The type of a function or method invocation is the type of its body. The type of expressions that have more than one branch (`if`) is the lowest common ancestor of the types of each branch, or ultimately `Object`.

A.9.3 Type inference of symbols

Once all expressions have been type-inferred, the type inferer will attempt to assing a type to each symbol declaration that is not explicitly annotated. Instead of providing an exact algorithm, we will define a set of constraints that the type inferer must satisfy whenever it succeeds in assigning a type.

Specific implementations of HULK can choose different methods to attempt the type inference of symbols. According to the order in which symbols are processed, and the sophistication of each method, some implementations may succed where others fail. However, if two type inference algorithms are correct, they most agree on all types for which both succeed in the inference.

These are the constraints a type inference algorithm must satisfy to be correct, or otherwise it must report a failed inference.

- In a `let` expression, whenever a variable is not type-annotated, the type inferer must asign a type for the variable that is equivalent to the type infered for its initialization expression.
- Similarly, in an attribute declaration that is not type-annotated, the type inferer must assign a type that is equivalent to the type inferred for its initialization expression.
- In a function or method, whenever an argument is not type-annotated, the type inferer must assign the lowest (most specific) type that would be consistent with the use of that argument in the method or function body. If more than one type in different branches of the type hierarchy would be consistent, the type inferer must fail.

- Similarly, in a type argument, the type inferer must assign the lowest type that is consistent with the use of that argument in all attribute initialization expressions where it is referenced.

If a type inferer satisfies those constraints, we will say it is *sound*. This means that, for example, the simplest sound strategy for type inference is to infer types for all expressions and fail for all symbols. We will call this the *basic inference* strategy.

A.9.4 Examples of ad-hoc type inference

These are some programs where a sufficiently sophisticated type inference strategy should work.

In the following program the type of the variable `x` should be inferred to `Number` because the type of `42` is trivially `Number`:

```
let x = 42 in print(x);
```

In the following function, the type of the argument `n` should be inferred as `Number` because it is the only possible type where arithmetic operators (i.e., `+`) are defined, as there is no operator overloading in HULK:

```
function fib(n) => if (n == 0 | n == 1) 1 else fib(n-1) + fib(n-2);
```

If you implement operator overloading, then the inferred type should be the appropriate protocol.

For the same reason, in the following function, the type of the argument `x` should be inferred as `Number`. Likewise, the type of the variable `f` should be inferred as `Number` because the initialization expression is a literal `Number`.

```
function fact(x) => let f = 1 in for (i in range(1, x+1)) f := f * i;
```

A.9.5 A general strategy for type inference

If you implement protocols (explained later), then a general strategy for type inference consists in synthesizing appropriate protocols for all non-annotated symbols, based on their use. Since protocols support structural type checking, this should allow the type checker to detect any inconsistencies in a later pass.

For example, consider the following code:

```
type A {
    f() => "Hello";
    g() => "World";
}

function h(x) => x.f() @@ x.g();

let x = new A() in print(h(x));
```

In the previous code, the type inferrer can determine that, whatever type `x` has, it should support two methods, `f` and `g`. Furthermore, given the use of the `@@` operator, the return value of both methods should support the `@@` operation (in principle, only `String` does, but if you implement operator overloading, there is a specific protocol for that operator).

Thus, the type inferrer can synthesize the following protocol:

```
protocol _P1 {
    f(): String;
    g(): String;
}
```

And it should annotate the code (actually, the AST) in a way that is equivalent to the following:

```
// type A and Protocol _P1

function h(x: _P1): String => x.f() @@ x.g();

let x: A = new A() in print(h(x));
```

From the point of view of the type checker, the previous code is semantically correct, since `A` conforms to the protocol `_P1`.

Note that the process of synthesizing protocols could require several iterations, since not all types in a synthesized protocol may be known at a first glance. For example:

```
function f(x) => x.a();

function g(x) => x.b();

let x = new T() in print(g(f(x)));
```

Regardless of how `T` looks like, the type inferrer here must first define a protocol for `f` that has a method `a()`, and analogous for `g`. But crucially, at this point, it is not clear from either `f` or `g` what the return type of these methods is.

Thus, at this point, the best a type inferrer can do is claim `f` receives something like:

```
protocol _P1 {
    a(): Any;
}

// ...

function f(x: _P1): Any => x.a();
```

And similarly for `g`:

```
protocol _P2 {
    b(): Any;
}

// ...

function g(x: _P2): Any => x.b();
```

Then, a series of passes on the AST start to refine these protocols. For example, the call to `g` in the last line of the code above will force the type inferrer to refine `_P1` to:

```
protocol _P1 {
    a(): _P2;
}
```

Which in turns, makes `f` now return `_P2`. Likewise, the call to `print` makes the type inferrer refine `_P2` to:

```
protocol _P2 {
    b(): Object;
}
```

Once now new information can be inferred, the type inferrer will stop and the program will be type checked. All types left as `Any` will be reported as errors.

> **NOTE:** To code a robust type inferrer is much harder than what the previous explanation might seem. There are plenty of corner cases and heuristics. This section is just an initial suggestion to guide the implementation.

A.10 Protocols

Protocols are special types which support a limited form of structural typing in HULK. The difference between structural and nominal typing in HULK, is that the latter is explicit while the former is implicitely defined. That is, a type doesn't need to explicitely declare that it conforms to a protocol.

Protocols have a syntax similar to that of types, except that they only have method declarations, and they have no body, only signatures. Hence, protocols define the methods that a type must have in order to support some operation.

Protocols don't exist at runtime, they are compile-time only concept that helps writing more flexible programs. After type checking, all information about protocols can be safely removed.

A.10.1 Defining protocols

A protocol is defined with the keyword `protocol` followed by a collection of method declarations:

```
protocol Hashable {
    hash(): Number;
}
```

A protocol can have any number of method declarations. For obvious reasons, all method declarations in protocol definitions must be fully typed, as it is impossible to infer any types since they have no body.

A protocol can extend anoter protocol by adding new methods, but never overriding (since there is no actual body) or removing any method (althought you can override the types of some method arguments or return types provided with some restrictions explained below).

```
protocol Equatable extends Hashable {
    equals(other: Object): Boolean;
}
```

A.10.2 Implementing protocols

A type implements a protocol implicitely, simply by having methods with the right signature. There is no need to explicitely declare which types implement which protocols.

Thus, you can annotated a variable or argument with a protocol type, and the type checker will correctly verify the consistency of both the method body and the invocation.

```
type Person {
    // ...

    hash() : Number {
        // ...
    }
}

let x : Hashable = new Person() in print(x.hash());
```

Anywhere you can annotate a symbol with a type (variables, attributes, function, method and type arguments, and return values), you can also use a protocol. For the purpose of type inference, protocols are treated as types.

A.10.3 Variance in protocol implementation

In order to implementing a protocol, a type doesn't necessarily have to match the exact signature of the protocol. Instead, method and type arguments are considered *contravariant*, and return values *covariant*. This means that arguments can be of the same type or higher, and the return values of the same type or lower than as defined in the protocol.

Similarly, when you extend a protocol, you can override some of the methods as long as you respect the variance constraints.

A.10.4 Conforming with protocols

More formally, protocols extend the notion of type conforming by adding the following rules:

- A type `T` conforms to a protocol `P` if `T` has all the method defined in `P` with the appropriate types (respecting the variance constraints explained before).
- If a protocol `P1` extends a protocol `P2`, then trivially `P1 <= P2`.
- A protocol `P1` also conforms to another protocol `P2` if any type that conforms to `P1` would also conform to `P2`, even if there is no explicit extension declared.

A.11 Iterables

An iterable in HULK is any object that follows the iterable protocol, which is defined as follows:

```
protocol Iterable {
    next() : Boolean;
    current() : Object;
}
```

An example of iterable is the builtin `range` function, which returns an instance of the builtin `Range` type, defined as follows:

```
type Range(min:Number, max:Number) {
    min = min;
    max = max;
    current = min - 1;

    next(): Boolean => (self.current := self.current + 1) < max;
    current(): Number => self.current;
}
```

Notice that since protocols are covariant in the return types of the methods, the `Range` type correctly implements the `Iterable` protocol.

A.11.1 Using iterables with the `for` loop

As explained in the loops section, the `for` loop works with the `Iterable` protocol, which means you can apply `for` on any instance of a type that implements the protocol.

In compile-time, `for` is transpiled to a code that is equivalent, but explicitely uses the `Iterable` protocol members.

For example, the code:

```
for (x in range(0,10)) {
    // code that uses `x`
}
```

Is transpiled to:

```
let iterable = range(0, 10) in
    while (iterable.next())
        let x = iterable.current() in {
            // code that uses `x`
        }
```

This transpilation guarantees that even though the `Iterable` protocol defines the `current` method with return type `Object`, when you use a `for` loop you will get the exact covariant type inferred in `x`.

As a matter of fact, due to the transpilation process, the `Iterable` protocol itself is not even necessary, since nowhere is a symbol annotated as `Iterable`. However, the protocol is explicitely defined as a builtin type so that you can explicitly use it if you need to annotate a method to receive a black-box iterable.

Keep in mind, thought, that when you annotate something explicitely as `Iterable`, you are effectively forcing the type inferrer to assign `Object` as the type of the iteration variable (`x` in this example). This is one of the reasons it is often better to let HULK infer types than annotating them yourself.

A.11.2 Typing iterables

Since in the `Iterable` protocol we can only define (at this point) the return value of `current()` as `Object`, it is cumbersome to type arguments of a function or method as `Iterable`, because doing so will force you to downcast the elements to a desired type.

For this reason, HULK allows a special syntax for typing iterables of a specific type `T` using the format `T*`:

```
function sum(numbers: Number*): Number =>
    let total = 0 in
        for (x in numbers)
            total := total + x;
```

What happens under the hood is that when you use of `T*` anywhere in a HULK program, the compiler will insert an implicit protocol definition that looks like this:

```
protocol Iterable_T extends Iterable {
    current(): T;
}
```

Since protocols can be extended by overriding some methods with the correct variance constraints, the previous code will compile correctly.

A.11.3 Implementing collections

The iterable protocols defined so far encapsulates the concept of making *a single iteration* over the sequence of elements. In contrast, most collection types you will define allow for multiple iterations, even simultaneously, over the same sequence of elements.

To accomodate for this kind of behaviour, we can define an *enumerable* protocol that simply provides one method to create an iterable for one specific iteration everytime that is needed:

```
protocol Enumerable {
    iter(): Iterable;
}
```

With this protocol defined, the `for` loop is extended such that, when used with an enumerable instead of directly an iterable, it will transpile to a slightly different code:

```
let iterable = enumerable.iter() in
    while (iterable.next())
        let x = iterable.current() in {
            // ..
        }
```

A.12 Vectors

The builtin vector type provides a simple but powerful abstraction for creating collections of objects of the same type. In terms of functionality, a vector is close to plain arrays as defined in most programming languages. Vectors implement the iterable protocol so they can be iterated with a `for` syntax.

Vectors in HULK can be defined with two different syntactic forms: explicit and implicit.

A.12.1 Explicit syntax

An explicit vector of `Number`, for example, can be defined as follows:

```
let numbers = [1,2,3,4,5,6,7,8,9] in
    for (x in numbers)
        print(x);
```

Because vectors implement the iterable protocol, you can explicitely find a `next` and `current` methods in case you ever need them. Besides that, vectors also have a `size(): Number` method that returns the number of items in the vector.

Vectors also support an indexing syntax using square brackets `[]`, as in the following example:

```
let numbers = [1,2,3,4,5,6,7,8,9] in print(numbers[7]);
```

A.12.2 Implicit syntax

An implicit vector can be created using what we call a generator pattern, which is always an expression.

Here's one example:

```
let squares = [x^2 | x in range(1,10)] in print(x);
// prints 2, 4, 6, 8, 10, ...
```

In general, the syntax has the form `[<expr> | <symbol> in <iterable>]`, where `<expr>` is run in a new scope where `symbol` is iteratively bound to each element in the vector.

A.12.3 Typing vectors

Since vectors are iterables, you can safely pass a vector as argument to method that expects an iterable:

```
function sum(numbers: Number*): Number =>
    let total = 0 in
        for (x in numbers)
            total := total + x;

let numbers = [1,2,3,4,5] in
    print(sum(numbers));
```

However, inside `sum` you cannot use the indexing operator `[]` or the `size` method, because the argument is typed as an iterable, and not explicitly as a vector. To fix this, HULK provides another special syntax for vectors, using the `T[]` notation:

```
function mean(numbers: Number[]): Number =>
    let total = 0 in {
        for (x in numbers)
            total := total + x;

        // here `numbers` is known to be vector
        total / numbers.size();
    };

let numbers = [1,2,3,4,5] in
    print(mean(numbers));
```

Like with iterables, what happens under the hood is that the compiler implicitely defines a type with the following structure:

```
type Vector_T {
    size() {
        // impementation of size ...
    }

    iter(): Iterable_T {
        // implementation of iter
    }
}
```

A.13 Functors

A functor in HULK is an object that encapsulates a function, which means it supports the `obj()` syntax. This can be accomplished with protocols easily, via transpilation. If you have a type that implements a functor protocol, then HULK will allow you to use the functor syntax. A functor protocol is any protocol that has an `invoke` method with appropriate type annotations.

For example, suppose you declare the following protocol in HULK:

```
protocol NumberFilter {
    invoke(x: Number): Boolean;
}
```

Then, you can annotate a function to receive an object that implements this protocol:

```
function count_when(numbers: Number*, filter: NumberFilter) {
    let total = 0 in
        for (x in numbers)
            total := total + if (filter.invoke(x)) 1 else 0;
}
```

But, since that protocol is a functor (it contains an `invoke` method), you can also use it directly as if it where a method, with the following syntax:

```
function count_when(numbers: Number*, filter: NumberFilter) {
    let total = 0 in
        for (x in numbers)
            total := total + if (filter(x)) 1 else 0;
}
```

To implement a functor protocol, you simply define a type that implements the protocol, as usual, and then you can use it:

```
type IsOdd {
    invoke(x: Number): Boolean => x % 2 == 0;
}

let numbers = range(0, 100) in
    print(count_when(numbers, IsOdd())); // prints `50`
```

But this syntax is extremely cumbersome, so HULK provides lots of syntax sugar to simplify the declaration and usage of functors.

A.13.1 Implicit functor implementation

The first aid that HULK provides is by implicitely implementing wrapping functions as functor types upong usage. For example, instead of defining the `IsOdd` type like before, you can simply define an `is_odd` function like the following, and pass it directly to the `count_when` function:

```
function is_odd(x: Number) => x % 2 == 0;

let numbers = range(0, 100) in
    print(count_when(numbers, is_odd));
```

And then HULK will automatically create an appropriate functor type that implements the desired protocol, which means the previous code is transpiled to something like the following:

```
function is_odd(x: Number) => x % 2 == 0;

type _IsOddWrapper {
    invoke(x: Number): Boolean => is_odd(x);
}

let numbers = range(0, 100) in
    print(count_when(numbers, _IsOddWrapper()));
```

Naturally, this syntax sugar extends to variable assignment as well, which means the following is valid:

```
let numbers = range(0, 100), filter: NumberFilter = is_odd in
    print(count_when(numbers, filter));
```

A.13.2 Lambda expressions

Keeping up with the previous example, we can eliminate the explicit `is_odd` definition and pass a lambda expression, which is an anonymous function defined directly in the place when the functor is needed:

```
let numbers = range(0, 100) in
    print(count_when(numbers, (x: Number): Boolean => x % 2 == 0));
```

The general syntax for lambda expressions is very similar to the syntax for inline functions, except that you don't need to name the function.

Also, if the type inferrer is good enough, you can almost always drop the explicit type annotations:

```
let numbers = range(0, 100) in
    print(count_when(numbers, (x) => x % 2 == 0));
```

And of course, lambda expressions can be stored in appropriately typed variables:

```
let numbers = range(0, 100), filter: NumberFilter = (x) => x % 2 = 0 in
    print(count_when(numbers, filter));
```

And the type inferrer is good enough, since `count_when` requires a `NumberFilter`, you can drop the explicit type annotation:

```
let numbers = range(0, 100), filter = (x) => x % 2 = 0 in
    print(count_when(numbers, filter));
```

A.13.3 Typing functors

And finally, we can also skip the protocol definition and use a special syntax for typing functors directly in the type annotaion:

```
function count_when(numbers: Number*, filter: (Number) -> Boolean) {
    // same code
}
```

The syntax `(Number) -> Boolean` indicates that we expect a functor with a single input of type `Number` and an output of type `Boolean`. Upon finding this definition, HULK will transpile that into something that is very similar to our explicit protocol definition:

```
protocol _Functor0 {
    invoke(_arg0: Number) : Boolean;
}

function count_when(numbers: Number*, filter: _Functor0) {
    // same code
}
```

A.14 Macros

Macros are a way to extend HULK with "functions" that are transpiled at compilation-time to standard HULK, instead of executed in runtime. But macros are considerable more powerful than functions, both sintactically and semantically. Macros in HULK are extremely powerful because they work at the sintactic level, which means they perform transformations directly over the abstract syntax tree. Besides that, their syntax allows to define sort of keyword-like language constructs.

Since macros are a complex topic, let's start with a simple scenario.

Suppose you want to have something like the following in HULK:

```
repeat(10) {
    // expressions
}
```

You quickly see that this code is equivalent to the (arguably a lot more verbose) following syntax:

```
let total = n in
    while (total >= 0) {
        total := total - 1;
        // expressions
    };
```

You can easily encapsulate this pattern in a `repeat` function that takes a number and an a general expression (as a functor):

```
function repeat(times: Number, expr: () -> Object): Object {
    let total = n in
        while (total >= 0) {
            total := total - 1;
            expr();
        };
}
```

And while this may work for your case, it has a couple of downsides. First, you don't exactly get the desired syntax, instead of:

```
repeat(10) {
    // expressions
}
```

You have to write something like the following, which is close, but still slightly more cumbersome and dirty.

```
repeat(10, () => {
    // expressions
});
```

The second, and most important one, is that the `expr` here encapsulates a computation that, from the point of view of the `repeat` function, is a black box. We will focus on why this matters later on.

A.14.1 Defining macros

Instead of a function, you can use a *macro*, which has a very similar syntax in HULK:

```
def repeat(n: Number, *expr: Object): Object =>
    let total = n in
        while (total >= 0) {
            total := total - 1;
            expr;
        };
```

But this change makes macros exceedingly more powerful than functions in a lot of cases, for a few reasons. First, notice the use of the `*expr: Object` syntax, instead of the `expr: () -> Object`. Here the `*` denotes that this `expr` is **not** a regular argument, instead it is a special argument that refers to the code inside the brackets *after* the macro invocation. Thus, you can use the following syntax:

```
repeat(10) {
    print("Hello World");
}
```

The `{ print("Hello World"); }` expression block is precisely what is passed on in the special argument `*expr`.

However, there is much more going on under that macro invocation. Instead of calling a functor in runtime, macros are expanded in compile time and transpiled into their bodies, which means there is no real `repeat` function anywhere in the compiled code. Instead, the actual code that is executed is something like:

```
let _total = 10 in
    while (_total >= 0) {
        _total := _total - 1;
        {
            print("Hello World");
        };
    }
```

This is the reason why you don't see `expr();` in the macro body, but `expr;`. That is, the body is not *executed* but *interpolated* inside the macro. This transpilation step makes macros often faster than functions because there is no extra overhead for passing arguments, however, you must be careful when thinking about the operational semantics of a macro especially where they differ from a regular function call.

A.14.2 Variable sanitization

Upon macro expansion, the variables inside the body of a macro are replaced with a special unique name generated by the compiler. This ensures that no variable in the context of the macro invocation can be accidentally hidden or used in unpredictable ways.

Take for example the following code:

```
let total = 10 in repeat(total) {
    print(total);
};
```

If variables inside the body of the `repeat` macro wheren't sanitazed, then the `print` statement would print `9`, `8`, etc, which is kind of unexpected unless you happen to know how the `repeat` macro is implemented, violating the principle of encapsulation. Even worse, this would happen if your variable is named `total`, but not if it's named something else, which again is surprising and inconsistent. However, since the variable `total` inside the body of `repeat` will be renamed to something completely different upon macro expansion, you can be certain that the `print` statement will work as expected, regardless of the name you happen to choose for your variable.

A.14.3 Symbolic arguments

There are times, though, when you want the macro to reuse a symbol that comes from its external context (a variable or attribute). In these cases, you can use the especial syntax `@symbol` to define a *symbolic argument* in the macro, and then bind a specific symbol upon macro expansion.

This is best explained with an example. Let's suppose we want to implement a `swap` macro that swaps the content of two variables. This cannot be done unless the macro can actually assign to the variables we want to swap. We would define the macro as:

```
def swap(@a: Object, @b: Object) {
    let temp: Object = a in {
        a := b;
        b := temp;
    }
}
```

And we invoke the macro as:

```
let x: Object = 5, y: Object = "Hello World" in {
    swap(@x, @y);
    print(x);
    print(y);
};
```

Which will be expanded to something like (except that `_temp` will be a generated name):

```
let x: Object = 5, y: Object = "Hello World" in {
    let _temp = x in {
        x := y;
        y := _temp;
    };
    print(x);
    print(y);
};
```

Notice how the actual names of the `x` and `y` variables are interpolated in the macro expansion. Of course, the type checker will guarantee that on invocation the `x` and `y` symbols are variables of the corresponding type.

A.14.4 Variable placeholders

Macros can also introduce a new symbol into the scope in which they are expanded, which can then be used in the body argument (or the other arguments). The syntax for this is `$symbol`. We call this a "variable placeholder", because it holds the name for a variable that will be introduced upon macro expansion.

Again, this is best explained with an example. Let's add a variable to the `repeat` macro to indicates the current iteration. We would define the macro as:

```
def repeat($iter: Number, n: Number, *expr:Object) {
    let iter: Number = 0, total:Number = n in {
        while (total >= 0) {
            total := total - 1;
            expr;
            iter := iter + 1
        };
    }
}
```

Now when calling the macro, you can specify a name for the `$iter` variable placeholder:

```
repeat(current, 10) {
    print(current);
};
```

The effect is that upon macro expansion, the variable placeholder `$iter` will be renamed to `current` and thus the body of the macro will correctly reference it. The actual expansion looks similar to the following code:

```
let current: Number = 0, _total:Number = n in {
    while (_total >= 0) {
        _total := _total - 1;
        {
            print(current);
        };
        current := current + 1
    };
};
```

The compiler ensures that the use of the new variable in the body of the macro is consistent with the type declared for the variable placeholder in the macro. However, it is entirely possible for the macro not to define the variable, or to define it conditioned on some structure of the body (we will see how that's achieved in the pattern matching section). In any case, since macro expansion is performed at compile time, any inconsistency that may arise will be captured by the compiler.

A.14.5 Pattern matching

By far the most powerful feature of macros is structural pattern matching. This feature allows to deconstruct an argument and generate a specific code depending on the argument *structure*. The reason this is possible is because macros run on compile time, so when you declare an argument of type `Number`, for example, what you'll get in the macro body is the actual expression tree of the argument, and not just the final evaluated object.

As everything else with macros, this feature is much better understood with examples. Let's suppose you want to define a macro called `simplify`, for no better use than to illustrate how powerful macros are compared to regular functions. This is how you would do it:

```
def simplify(expr:Number) {
    match(expr) {
        case (x1:Number + x2:Number) => simplify(x1) + simplify(x2);
        case (x1:Number + 0) => simplify(x1);
        case (x1:Number - x2:Number) => simplify(x1) + simplify(x2);
        case (x1:Number - 0) => simplify(x1);
        case (x1:Number * x2:Number) => simplify(x1) * simplify(x2);
        case (x1:Number * 1) => simplify(x1);
        // ... you get the idea
        default => expr;
    };
}
```

You would use the macro as follows:

```
print(simplify((42+0)*1));
```

And the actual generated code would be:

```
print(42);
```

Notice that this transformation happens during compilation time, not execution. The actual code that gets compiled is the simplified expression.

B The BANNER Intermediate Representation

Every high-level abstraction is a well-intentioned lie told to the programmer to shield them from the cold, mechanical reality of the hardware. When we write in HULK, we inhabit a world of rich objects, nested expressions, and elegant recursion—a world where the complexity of the machine is hidden behind a veil of syntactic grace. However, the silicon upon which this logic eventually runs is fundamentally indifferent to such elegance. To a processor, there are no "objects" with "methods," nor are there "types" in the way we understand them; there is only memory, registers, and a relentless sequence of primitive operations. The BANNER Intermediate Representation (IR) is the site of the great reconciliation—it is Phase 1 of a structural audit where high-level intent is methodically stripped of its finery and translated into the explicit, linear, and minimalist language of raw execution.

The transition from a language as expressive as HULK to raw machine code is too steep a cliff to be traversed in a single leap. Direct compilation would force the compiler to manage complex tasks simultaneously: register allocation and stack frame management would have to be handled while simultaneously unraveling deep semantic structures like inheritance hierarchies and dynamic dispatch. BANNER exists to decouple these concerns. By providing a "Three-Address Code" (3AC) architecture, it offers a representation that is close enough to the machine to be easily translated into assembly or bytecode, yet abstract enough to remain portable and amenable to systematic optimization.

In the minimalist world of BANNER, the lush landscapes of HULK are flattened into a linear sequence of explicit instructions. Every complex mathematical expression is decomposed into a series of simple assignments involving temporary variables, ensuring that no instruction ever performs more than one fundamental operation. Control flow structures like `if-else` blocks and `while` loops are stripped of their structured sugar and reduced to the raw mechanics of labels and conditional jumps. This "flattening" process is not merely a simplification; it is a rigorous accounting of every operation the CPU must eventually perform. In BANNER, the ambiguity of high-level scope is replaced by the absolute clarity of `GOTO` and `LABEL`.

Perhaps the most significant shift in BANNER is the loss of high-level type safety in favor of a "everything is a number" philosophy. While HULK enforces a strict type system, BANNER treats all values as 32-bit integers, where the meaning of a value is defined entirely by how it is used. An integer might be a literal constant, a memory address, or a pointer to a virtual method table. This transparency reveals the true cost of object-oriented programming: attribute access becomes a calculated offset, and a method call becomes a dynamic lookup. By enforcing this level of explicitness, BANNER allows the compiler to perform optimizations that would be

impossible at a higher level, serving as the indispensable foundation upon which the final binary is built.

B.1 The Anatomy of a BANNER Program

A BANNER file is not merely a list of instructions; it is a structured blueprint that organizes the entire memory and logic of a program into three distinct, top-down sections. This organization reflects the fundamental pillars of an object-oriented runtime: the definition of object layouts, the management of static resources, and the execution of procedural logic. By separating these concerns into `.TYPES`, `.DATA`, and `.CODE` blocks, BANNER provides a clear roadmap for how high-level HULK abstractions are physically mapped onto the computer's memory.

B.1.1 The `.TYPES` Section: Flattening the Hierarchy

The `.TYPES` section is where the rich, recursive world of HULK classes is reduced to linear memory layouts. In HULK, a class might inherit from multiple ancestors, but in BANNER, all inheritance is resolved. Each entry in the `.TYPES` section defines a unique object structure, listing every attribute—including those inherited—in a fixed order. This ensures that an attribute like `x` always appears at the same memory offset relative to the object's start address, regardless of whether it was defined in a base class or a specialized subclass.

Beyond attributes, the `.TYPES` section maps method names to specific function labels in the `.CODE` section. This is the foundation of dynamic dispatch. When a method is overridden in a subclass, the `.TYPES` entry for that subclass simply points the method name to a different function label. This explicit mapping turns the abstract concept of "method lookup" into a simple pointer redirection.

Example: Consider a HULK class `A` and its subclass `B`. In BANNER, their layouts are explicitly defined to preserve structural compatibility:

```
.TYPES

type A {
    attribute x ;
    method f : f1;
}

type B {
    attribute x ;   # Inherited from A, same offset
    attribute y ;   # New attribute
    method f : f2 ; # Overridden method: points to f2 instead of f1
```

```
    method g : f3 ; # New method
}
```

B.1.2 The .DATA Section: The Static Pool

While most data in HULK is dynamic, certain values—most notably strings—are constant and known at compile time. The `.DATA` section serves as a global string pool. In BANNER, strings are treated as immutable blocks of memory. This section ensures that every unique string literal used in the program is allocated once and can be referenced by a label. Each entry in `.DATA` associates a human-readable label with a literal value, allowing the code to reference these resources by name rather than hard-coded memory addresses.

Example:

```
.DATA

msg = "Hello World";
error_msg = "An error has occurred";
```

B.1.3 The .CODE Section: Procedural Execution

The heart of the program resides in the `.CODE` section, which contains the actual implementation of every function and method. Unlike HULK, where functions can be nested and capture variables from their environment, BANNER functions are strictly top-level entities. Each function follows a rigid internal structure: it first declares its `PARAM` variables (inputs from the caller), then its `LOCAL` variables (scratchpad memory for the function's own use), and finally its sequence of 3-address instructions.

In this minimalist environment, there is no automatic scope management. Every temporary value used in a complex calculation must be explicitly declared as a `LOCAL` variable. This explicitness turns the implicit stack management of HULK into an observable, linear process.

Example: An implementation for the functions referenced in the `.TYPES` section might look like this:

```
.CODE

function f1 {
    PARAM self ;     # Explicit 'self' parameter
    PARAM x ;

    LOCAL a ;        # Local scratchpad
```

```
    LOCAL b ;

    # ... instructions ...
    RETURN a ;
}

function f2 {
    PARAM self ;
    # ... overridden logic ...
    RETURN 0 ;
}
```

By the time the compiler reaches the end of a BANNER program, the high-level intent of the programmer has been fully cataloged: the objects are measured, the constants are pooled, and the logic is linearized. This structural clarity is what makes BANNER an ideal bridge between the abstract and the mechanical.

B.2 The Instruction Set: A Minimalist Vocabulary

The elegance of HULK's expression-based syntax is nowhere to be found in the BANNER instruction set. Instead, we are left with a sparse collection of primitives that reflect the iterative, step-by-step nature of physical execution. Every operation is explicit, every memory access is calculated, and every jump is absolute. By reducing the language to these few atomic actions, we ensure that the final translation to machine code is a systematic mapping of BANNER instructions to their CPU equivalents.

B.2.0.1 Data Movement and Arithmetic

At its most basic level, a program is a sequence of transformations applied to data. In BANNER, these transformations are expressed through three-address assignments—a format where every instruction has at most two operands and one result. No matter how complex a mathematical expression might be in HULK, it must be decomposed into a series of operations where a single operator is applied to its inputs.

Example: The HULK expression `z = (x + y) * 2` cannot be represented as a single instruction. It must be broken down into discrete steps using temporary local variables to hold intermediate results—preserving the strict three-address format required by the IR.

```
t1 = x + y ;
t2 = t1 * 2 ;
z = t2 ;
```

B.2.0.2 Memory Management

In a high-level language, memory management is often "invisible"—objects simply appear when needed and disappear when they are no longer reachable. In BANNER, the creation of every object and array is a deliberate act that must be explicitly requested from the runtime. This explicitness forces the compiler to account for the physical reality of heap allocation.

Example: When a programmer instantiates a class or creates a fixed-size buffer, the BANNER representation uses `ALLOCATE` to reserve space for a structured type or `ARRAY` to request a contiguous block of memory—returning a pointer that will be treated as a 32-bit integer.

```
p = ALLOCATE Point ;  # Reserves memory for a 'Point' instance
a = ARRAY 10 ;        # Reserves a block for 10 elements
```

B.2.0.3 Object Interaction

Once an object is allocated, interacting with its internal state requires direct manipulation of its memory layout. BANNER does not understand high-level properties; it understands memory offsets relative to a base address. The `GETATTR` and `SETATTR` instructions are the primary tools for reading from and writing to the fields defined in the `.TYPES` section.

Example: Updating the `x` coordinate of a `Point` object involves identifying the correct attribute label—which the backend eventually translates to a numerical offset—and performing a store operation. Reading that value back requires a corresponding load into a local variable.

```
val = 42 ;
SETATTR p Point_x val ; # Writes 42 into the 'x' attribute of object 'p'
curr = GETATTR p Point_x ; # Reads the value back into 'curr'
```

B.2.0.4 Control Flow

High-level control structures like `if` statements and `while` loops are essentially "syntactic sugar" for conditional and unconditional jumps. BANNER strips away this structure, relying instead on a flat system of labels and jumps. This mimics how a CPU's instruction pointer moves through memory—branching only when specific conditions are met.

Example: A conditional check is implemented by evaluating a predicate and then using `IF ... GOTO` to jump to a specific label. If the condition is not met, execution simply continues to the next instruction—effectively creating the "else" or "exit" logic.

```
LABEL check_zero ;
IF x GOTO non_zero ;
PRINT "is zero" ;
GOTO end ;
LABEL non_zero ;
PRINT "is not zero" ;
LABEL end ;
```

B.2.0.5 The Call Stack and Method Invocation

The most complex part of BANNER is the management of function calls and dynamic dispatch. Since BANNER is a flat language, it must explicitly handle the passing of arguments and the retrieval of return values. This is achieved through a sequence of `PARAM` instructions that prepare the environment before a `CALL` (for static functions) or `VCALL` (for virtual methods) is executed.

Example: Invoking a method `move(dx, dy)` on a `Point` object requires passing the object itself—the `self` pointer—followed by its arguments. A `VCALL` is then used to look up the correct function implementation in the object's virtual method table based on the provided type.

```
PARAM p ;           # Pass the object instance (self)
PARAM 10 ;          # Pass dx
PARAM 20 ;          # Pass dy
r = VCALL Point move ; # Perform dynamic dispatch for 'move'
RETURN r ;          # Send the result back to the caller
```

B.3 Case Study: From HULK to BANNER

The transformation from HULK's high-level abstractions to the minimalist environment of BANNER is best understood as a structural audit. It is a process that strips away the syntactic elegance of the source language to reveal the explicit mechanical steps required for execution. This lowering process involves three primary tasks: decomposing nested expressions into linear three-address instructions, mapping class hierarchies into flat memory layouts, and converting implicit behaviors—like method dispatch and attribute access—into explicit calculations.

To illustrate this transformation, consider a classic "Hello World" scenario implemented using a class structure in HULK. This example highlights how objects are managed and how strings are handled as static resources.

B.3.0.1 The HULK Source

Consider the following HULK program. It defines a `Main` class with a single attribute and a method that prints that attribute, followed by an instantiation and a method call:

```
type Main {
    msg: String = "Hello World";
    run() => print(this.msg);
}

let m = new Main() in m.run();
```

At the Abstract Syntax Tree (AST) level, this program is a collection of nested nodes representing the class definition, attribute initialization, and a `Let` expression containing a `MethodCall`. To the BANNER compiler, this must be systematically dismantled and redistributed across the `.TYPES`, `.DATA`, and `.CODE` sections.

B.3.0.2 Step 1: Mapping the Static Layout

The first step is addressing the program's static requirements. The compiler identifies the `Main` class and determines its physical memory layout. In the `.TYPES` section, `Main` is registered with its attribute `msg` and its method `run`. Simultaneously, the string literal `"Hello World"` is extracted and placed in the `.DATA` section with a unique label, such as `s0`.

```
.TYPES
type Main {
    attribute Main_msg ;
    method run : Main_run ;
}

.DATA
s0 = "Hello World" ;
```

This separation is crucial: the object instance in memory will not contain the string itself, but rather a 32-bit reference (a pointer) to the address labeled `s0` in the data pool.

B.3.0.3 Step 2: Lowering the Entry Point

The logic found in the global scope of the HULK program (the `let` expression) is lowered into a special `entry` function in the `.CODE` section. Here, the "three-address" nature of BANNER

becomes visible. The compiler generates temporary local variables (often prefixed with `t`) to hold intermediate results.

The instantiation `new Main()` is transformed into an `ALLOCATE` instruction, and the attribute initialization is handled via `SETATTR`. Notice that the string must be explicitly "loaded" into a temporary variable before it can be assigned to the object.

```
function entry {
    LOCAL m ;
    LOCAL t1 ;
    LOCAL t2 ;

    t1 = LOAD s0 ;          # Load string address from .DATA
    m = ALLOCATE Main ;     # Reserve heap space for Main
    SETATTR m Main_msg t1 ; # Initialize 'msg' attribute

    PARAM m ;               # Pass the 'self' pointer
    t2 = VCALL Main run ;   # Perform dynamic dispatch
    RETURN 0 ;
}
```

B.3.0.4 Step 3: Implementing the Method

Finally, the `run` method itself is lowered. In HULK, `this` is an implicit reference to the current object. In BANNER, this becomes an explicit first parameter named `self`. The access to `this.msg` is transformed into a `GETATTR` operation, using the label `Main_msg` to determine the correct memory offset. The call to the built-in `print` function is then lowered into a primitive `PRINT` instruction.

```
function Main_run {
    PARAM self ;
    LOCAL l_msg ;

    l_msg = GETATTR self Main_msg ; # Extract the string reference
    PRINT l_msg ;                   # Send reference to the IO system
    RETURN self ;
}
```

By the end of this process, every high-level "magic" feature—be it inheritance, dynamic dispatch, or automatic string management—has been reduced to a sequence of explicit, manageable operations. This granularity is what allows the compiler to perform final optimizations and eventually generate the binary code that the hardware can execute.

B.4 Technical Deep Dive: "Everything is a Number"

At the heart of the BANNER architecture lies a radical commitment to architectural minimalism: the "everything is a number" philosophy. In the high-level world of HULK, developers reason about complex types—strings, boolean flags, and polymorphic class instances—but as these abstractions descend into the BANNER Intermediate Representation, they are stripped of their semantic metadata and reduced to a uniform 32-bit integer format. This homogenization is not merely a technical convenience; it is a fundamental design choice that aligns the IR with the mechanical reality of the hardware, where the distinction between a memory address, a numerical literal, and a bitmask is entirely a matter of perspective.

In this environment, the meaning of a value is not inherent to the value itself but is instead derived from the context of its usage—a concept we might call *contextual semantics*. When a BANNER instruction performs an arithmetic operation like `x = y + z`, the virtual machine treats the operands as raw numerical data to be manipulated by the ALU. However, when the same variables appear in a memory-oriented instruction like `x = GETATTR y z`, the interpretation shifts dramatically: `y` is suddenly treated as a pointer to a base address in the heap, while `z` is interpreted as a numerical offset within that object's structure. This flexibility allows for a highly compact instruction set, but it requires that the compiler maintains an absolute, unwavering map of what every number "represents" at any given point in execution.

This design shift fundamentally reallocates the burden of semantic safety from the runtime to the compiler's semantic analyzer. Unlike more "helpful" virtual machines—such as the JVM or the Python interpreter—the BANNER VM performs no runtime type-checking or safety audits on its instructions. If a compiler emits a `GETATTR` call on a value that is actually a string literal or a mathematical constant, the VM will dutifully attempt to dereference that memory location, likely resulting in a segmentation fault or the retrieval of garbage data. By removing these safety checks from the execution loop, BANNER achieves a level of performance closer to native code, operating on the assumption that the preceding compilation phases have already "proven" the correctness of the instruction stream.

For the Rust-based backend, this minimalist approach creates a fascinating dichotomy of efficiency and complexity. On one hand, the VM's core execution loop is exceptionally lean, as it can leverage Rust's primitive integer types and direct memory access without the overhead of constant type-tagging or dynamic dispatch overhead. On the other hand, it necessitates a sophisticated heap management and garbage collection system. Since the VM itself cannot distinguish a pointer from a literal 32-bit integer, the backend must employ advanced techniques—such as shadow stacks or pointer tagging—to ensure that the garbage collector can safely identify reachable objects without accidentally "collecting" a valid memory address that looks like a large number. This tension between IR simplicity and backend robustness is the defining characteristic of the BANNER ecosystem.

B.5 Conclusion: The Unseen Foundation

The journey from the high-level elegance of HULK to the minimalist, three-address world of BANNER represents more than just a technical translation; it is a fundamental shift in how we conceptualize computation. BANNER serves as what we might call "assembly for the mind"—a simplified yet remarkably powerful model that captures the essence of execution without the suffocating complexity of physical hardware. By stripping away the abstractions of classes, inheritance, and nested expressions, BANNER reveals the underlying mechanical logic that drives every software system. It is here, in this intermediate space, that the true architecture of a program is laid bare, offering a level of clarity that is often lost in the dense syntax of high-level languages or the cryptic mnemonics of machine code.

From a pragmatic perspective, BANNER is the architectural pivot point that enables both optimization and portability. By decoupling the front-end analysis of HULK from the back-end details of the machine, BANNER provides a stable platform for intermediate passes. It is at this stage that a compiler can perform dead-code elimination, constant folding, and common subexpression elimination with surgical precision, independent of whether the final target is an x86 processor, an ARM chip, or a custom virtual machine. This separation of concerns is the hallmark of professional compiler design, ensuring that the "what" of the programmer's intent is preserved while the "how" of its execution is refined for maximum efficiency.

Ultimately, BANNER is the final, indispensable piece of the HULK-to-Machine puzzle. It is the bridge that spans the chasm between human-readable intent and silicon-executable instructions. Without this intermediate foundation, the task of building a compiler would be an overwhelming exercise in managing conflicting complexities. With BANNER, however, the process becomes a series of manageable, logical steps. It serves as a reminder that even the most sophisticated systems are built upon simple, atomic foundations—and that understanding these foundations is the key to mastering the art of software engineering.

As you move forward into the implementation of the backend and the nuances of the garbage collector, keep the minimalist spirit of BANNER in mind. It is a testament to the idea that power does not always come from complexity, but often from the rigorous application of simple rules. BANNER invites us to look beneath the surface of our high-level tools and appreciate the unseen machinery that makes modern computing possible. In the end, the most robust foundations are often those that remain hidden, quietly supporting the weight of the abstractions we build upon them.

C Tooling for HULK

While the core of HULK is its compiler and runtime, a modern programming language also requires a robust set of tools to support developers. This appendix outlines the high-level design of the HULK tooling ecosystem, focusing on editor integration and language support.

C.1 Editor Support and Syntax Highlighting

Syntax highlighting is the first step toward a good developer experience. For HULK, this is achieved through TextMate grammars and custom XML definitions.

C.1.1 TextMate Grammars

HULK's syntax is primarily defined in `vscode/syntaxes/hulk.tmLanguage.json`. This JSON file uses regular expressions to categorize language constructs: - **Keywords**: Control flow (`if`, `else`, `while`, `for`), declarations (`function`, `type`, `let`, `protocol`, `def`). - **Literals**: Numbers, strings, and booleans. - **Operators**: Arithmetic, boolean, assignment (`:=`), and special macro operators (`@`, `$`).

To support HULK code blocks in Markdown and Quarto files, a specialized grammar `hulk-markdown.tmLanguage.json` is used to inject HULK highlighting into fenced code blocks.

C.1.2 Quarto Integration

For the static rendering of this book, Quarto uses a custom XML syntax definition (`syntax/hulk.xml`) that mirrors the rules of the TextMate grammar. This allows code blocks marked with "'hulk to be highlighted in the generated HTML and PDF formats.

C.2 Language Server Protocol (LSP)

The Language Server Protocol provides a standard way for editors to communicate with language-specific backends. A HULK LSP would ideally provide the following features:

C.2.1 Diagnostics

As the developer types, the HULK compiler should run in the background to report syntax errors and type mismatches. These appear as real-time feedback (e.g., red squiggly lines) in the editor.

C.2.2 Navigation and Go to Definition

By traversing the symbol table created during the semantic analysis phase, the LSP can provide "Go to Definition" for functions, variables, and types, allowing for easier navigation of large codebases.

C.2.3 Hover Information

Hovering over an identifier can show its inferred type or documentation. Since HULK features advanced type inference, the LSP reveals the exact type the compiler has deduced for a given variable.

C.3 VS Code Extension Development

The HULK VS Code extension (located in the `vscode/` directory) serves as the primary integration point. Its main components are:

1. **`package.json` Contributions**: Defines the `hulk` language ID, associated file extensions (`.hulk`), and pointers to grammars and snippets.
2. **Language Configuration**: Specifies comment symbols (`//`), bracket pairs, and indentation rules.
3. **Snippets**: Pre-defined templates for common HULK constructs like `function`, `type`, and `let` (located in `vscode/snippets/hulk.code-snippets`).
4. **LSP Client (Future Work)**: A TypeScript component that launches the HULK compiler in "LSP Mode" and manages communication between the editor and the compiler.

By following this architecture, HULK becomes not just a didactic language, but a well-integrated development environment.

D The Instructor's Manual